Image-to-LaTeX Mathematical Formula Recognition with Qwen2.5-VL Fine-Tuning

Hu Chenzhi, Wang Yaoyao, and Ming Jie Lim
Shanghai Jiao Tong University

12 May, 2025

Abstract

Image-to-LaTeX mathematical formula recognition is challenging because formulas contain complex two-dimensional layouts, nested symbolic structures, and large visual variation between printed and handwritten inputs. In this paper, we fine-tune Qwen2.5-VL-3B for formula image recognition and generation of LaTeX markup. To improve robustness and scale, we curate a training corpus of approximately 1 million samples by combining Im2LaTeX-100K with additional formula-image pairs collected from arXiv and Wikipedia, and we further enrich the dataset with handwritten formulas from CROHME and HME100K, which account for roughly 30% of the final corpus. We apply AST-based LaTeX normalization to reduce syntactic redundancy and improve target consistency during training. Experiments evaluate recognition quality using BLEU, exact match accuracy, and character error rate. The fine-tuned model substantially outperforms the untuned baseline, with the best configuration achieving BLEU scores around 0.896 while preserving strong output completeness. Among the tested inference settings, the unquantized model with SDPA provides the best overall trade-off between recognition accuracy and inference efficiency.

1 Introduction

Mathematical expression recognition is a core component of scientific editing tools, digital note-taking systems, and educational software. Unlike standard text OCR, mathematical expressions exhibit strong two-dimensional structure, including superscripts, subscripts, fractions, radicals, matrices, and nested layouts. These properties make formula recognition difficult for traditional OCR pipelines that separate symbol detection, classification, and structural parsing.

End-to-end neural models have improved this situation by directly mapping formula images to markup sequences. Early encoder-decoder systems such as Im2LaTeX [9] demonstrated the viability of image-to-markup generation, and more recent vision-language models have made multimodal generation substantially stronger. However, two issues remain central in practice: first, recognition quality still drops on complex layouts and ambiguous LaTeX forms; second, strong multimodal models often carry prohibitive inference cost for edge-side deployment.

The main contributions of this paper are threefold:

- It adapts Qwen2.5-VL-3B to image-to-LaTeX generation with task-specific supervised fine-tuning and normalized LaTeX targets.
- It studies the trade-offs among quantization, SDPA acceleration, quality, inference latency, and memory footprint.
- It outlines an end-to-end deployment workflow for local formula recognition using GGUF, Ollama, and Open WebUI.

2 Related Work

Classical formula recognition systems relied on multi-stage OCR pipelines, which are difficult to scale to diverse structures and writing styles. Neural encoder-decoder architectures improved robustness by learning

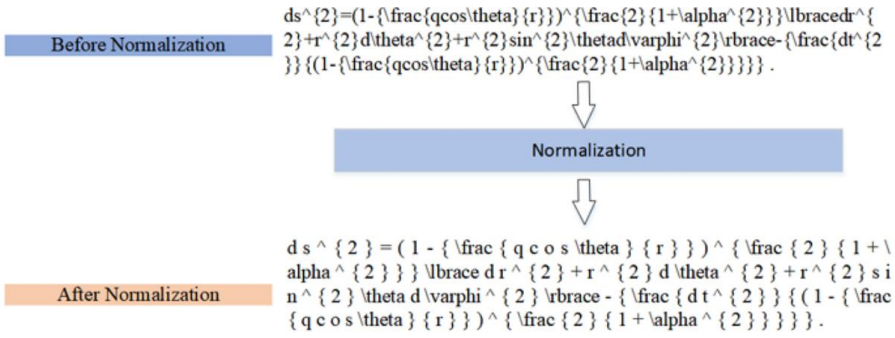

Figure 1: LaTeX normalization via AST-based canonicalization.

image-to-sequence mappings directly. Transformer-based models further advanced the field through stronger global context modeling [1]. In the vision domain, ViT [2] and Swin Transformer [3] established scalable image encoding paradigms. In multimodal modeling, Donut [4], LLaVA [8], and Qwen-VL style systems [5] showed that OCR-free generation can be effective for visually grounded language tasks.

For adaptation, parameter-efficient methods such as LoRA [6] and QLoRA [7] reduce training cost, but when the objective is maximal recognition quality, more direct supervised fine-tuning remains attractive. The present study follows that direction while freezing the visual encoder to reduce memory pressure.

3 Method

3.1 Base Model

We build on Qwen2.5-VL-3B [5], a multimodal Transformer that combines a visual encoder, a vision-language projection module, and a large language model decoder. Its visual stack uses a redesigned Vision Transformer with 2D rotary positional embedding and window attention, while an MLP-based projection compresses grouped visual tokens before they are passed to the language model.

This design is well suited to image-to-LaTeX generation because it treats formula recognition as a single multimodal sequence modeling problem rather than as a cascade of OCR and structural heuristics.

3.2 Dataset and Normalization

Our training corpus is curated from multiple sources rather than relying only on the original Im2LaTeX-100k [9] benchmark. We start from Im2LaTeX-100k and expand it with additional formula-image pairs collected through arXiv and Wikipedia scraping, then inject handwritten formula samples from CROHME and HME100K to improve robustness to non-printed inputs. In the final dataset, handwritten examples account for roughly 30% of the corpus, and the full training set is expanded to approximately 1 million records.

To reduce target-side variation, we normalize LaTeX strings before training. The preprocessing pipeline inserts explicit token spacing, removes redundant brackets, filters malformed expressions, and canonicalizes symbol and function forms with the help of AST-based parsing.

This normalization is important because many mathematically equivalent LaTeX sequences differ only in syntax. Without normalization, the model must learn a larger output space than necessary and evaluation becomes noisier.

3.3 Fine-Tuning Setup

Training is performed on four NVIDIA A800 GPUs with 80 GB memory each. The software stack uses PyTorch 2.5.0, Transformers 4.51.3, and Accelerate. We use bfloat16 precision, AdamW optimization, learning rate 1×10^{-5}, cosine decay, warmup ratio 0.1, weight decay 0.01, and two training epochs.

We freeze the ViT encoder and fine-tune the remaining multimodal model under supervised learning. Each GPU uses batch size 1 with gradient accumulation of 16, yielding an effective batch size of 16. Gradient checkpointing is enabled to control activation memory. Input formatting follows ChatML so that the model receives a consistent multimodal instruction-response structure.

The final run processes approximately 17.87 million tokens in 48,982 seconds, reaching validation loss 0.271 and mean training token accuracy 98.7%.

4 Experimental Setup

4.1 Metrics

We evaluate predictions with BLEU [10], exact match accuracy, and character error rate. These metrics capture complementary aspects of image-to-LaTeX quality and are therefore interpreted jointly rather than in isolation.

BLEU. BLEU measures the overlap between an output sequence H and a reference sequence R through n-gram precision with a brevity penalty:

$$\text{BLEU-}n = BP \cdot \exp\left(\sum_{i=1}^{n} w_i \log p_i\right), \tag{1}$$

where p_i denotes the i-gram precision, w_i is the corresponding weight, and BP is the brevity penalty. In our setting, BLEU reflects overall structural and lexical similarity between the generated LaTeX string and the target expression.

Exact match accuracy. Exact match accuracy measures the fraction of samples whose predicted LaTeX string is identical to the reference at the sequence level:

$$\text{Exact Match Accuracy} = \frac{M}{N}, \tag{2}$$

where M is the number of exactly matched predictions among N test samples. This is a strict metric and is particularly relevant when downstream applications require fully correct LaTeX output.

Character error rate. CER measures the normalized edit distance between the predicted and reference strings:

$$\text{CER} = \frac{S + D + I}{N}, \tag{3}$$

where S, D, and I denote the numbers of substitutions, deletions, and insertions, respectively, and N is the number of reference characters. CER is useful for identifying fine-grained token-level errors even when the full expression is not exactly matched.

None of these metrics fully resolves semantic equivalence in LaTeX, since two syntactically different expressions may still represent the same mathematical meaning. We therefore use all three metrics together to obtain a more balanced evaluation.

4.2 Deployment-Oriented Comparisons

The project compares the untuned model against four deployment-relevant settings: 4-bit double quantization, 8-bit quantization, no quantization, and no quantization plus SDPA. This allows us to evaluate the trade-off between generation quality and deployment cost.

Table 1: Quality and efficiency comparison across inference settings.

Setting	BLEU	Length Ratio	Time (s)	Memory (MB)
Untuned baseline	0.1658	0.5509	0.8212	7754.88
4-bit double quantization	0.3438	0.8821	2.3477	3878.00
8-bit quantization	0.8954	0.9842	7.9428	4025.55
No quantization	0.8959	0.9836	2.3151	7754.88
No quantization + SDPA	0.8964	0.9829	2.0987	7754.88

5 Results

5.1 Qualitative Analysis

The source project includes a representative qualitative case where the untuned baseline fails to reconstruct the full fraction structure and misparses subscript variables, while the fine-tuned model matches the ground-truth LaTeX exactly. This suggests that fine-tuning improves the model's ability to preserve hierarchical symbolic structure rather than only token-level local patterns.

5.2 Recognition and Efficiency Trade-Offs

Table 1 shows that fine-tuning is the dominant factor behind improved quality. The untuned model is fast but unusable, producing short and incomplete outputs. Four-bit double quantization halves memory but still leaves the system well below acceptable quality. By contrast, the unquantized and 8-bit settings achieve BLEU near 0.896, showing that the adapted model can accurately reconstruct target formulas.

Among the high-quality settings, the best overall trade-off is the unquantized plus SDPA variant. It slightly improves BLEU over the plain unquantized model and reduces inference time from 2.3151 to 2.0987 seconds without additional memory cost. The 8-bit model preserves quality but incurs a large latency increase to 7.9428 seconds, making it a poor edge deployment choice in the reported setup.

6 Deployment Discussion

To support local use on personal computers, the project converts trained checkpoints from `safetensors` to GGUF using llama.cpp-compatible tooling. The GGUF files can then be served with Ollama and interacted with through Open WebUI. This stack lowers the barrier to practical use and provides a reproducible path from training to desktop inference.

The experiments also reveal a remaining limitation: the best-performing setup still requires roughly 7.8 GB of memory. This is workable on high-end consumer devices but may still be too heavy for tablets or low-power edge hardware. Further compression, distillation, or low-rank adaptation remains necessary for broader deployment.

7 Limitations

This study inherits several limitations from the source project. First, evaluation is based largely on string-level metrics and does not fully capture semantic equivalence among valid LaTeX forms. Second, the dataset is dominated by rendered formulas rather than truly diverse handwritten input. Third, the deployment analysis is limited to a small set of quantization and attention configurations and does not yet include stronger lightweight baselines.

8 Conclusion

Fine-tuning Qwen2.5-VL-3B for image-to-LaTeX recognition substantially improves formula reconstruction quality relative to the untuned baseline. AST-based normalization reduces output variability, supervised mul-

timodal adaptation improves structural fidelity, and SDPA provides a favorable inference-speed improvement under the best-performing unquantized configuration. The deployment workflow through GGUF, Ollama, and Open WebUI further shows that high-quality LaTeX OCR can move closer to practical local use. Future work should focus on handwriting robustness, semantic evaluation, and lighter-weight deployment strategies.

References

[1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.

[2] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.

[3] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021.

[4] Geewook Kim, Teakgyu Hong, Moonbin Yim, Jinyoung Park, Jinyeong Yim, Wonseok Hwang, Sangdoo Yun, Dongyoon Han, and Seunghyun Park. Donut: Document understanding transformer without OCR. *arXiv preprint arXiv:2111.15664*, 2021.

[5] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibo Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, Humen Zhong, Yuanzhi Zhu, Mingkun Yang, Zhaohai Li, Jianqiang Wan, Pengfei Wang, Wei Ding, Zheren Fu, Yiheng Xu, Jiabo Ye, Xi Zhang, Tianbao Xie, Zesen Cheng, Hang Zhang, Zhibo Yang, Haiyang Xu, and Junyang Lin. Qwen2.5-VL technical report. 2025.

[6] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

[7] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems*, 2023.

[8] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. In *Advances in Neural Information Processing Systems*, 2023.

[9] Yuntian Deng, Anssi Kanervisto, Jeffrey Ling, and Alexander M. Rush. Image-to-markup generation with coarse-to-fine attention. In *International Conference on Machine Learning*, 2017.

[10] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. BLEU: A method for automatic evaluation of machine translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, 2002.